

RestOps — Full Gap Analysis (Spec vs Current Code)

Legend:  have it  partial/needs rework  missing entirely

1. Multi-tenancy (restaurant scoping)

Item	Status	Notes
<code>Restaurant</code> model		Doesn't exist. Everything currently assumes one global restaurant.
<code>restaurantId</code> on <code>User</code>		Needed so staff/manager are scoped to their restaurant
<code>restaurantId</code> on <code>Table</code>		Needed so tables/layout are scoped
<code>restaurantId</code> on <code>Order</code> , <code>Menu</code> , <code>WaitingQueue</code>		Every query needs to filter by this eventually
Manager = owner of a <code>Restaurant</code>		Need <code>Restaurant.owner</code> ref, and register-restaurant flow

This is foundational — almost everything below assumes this exists.

2. Auth & Roles

Item	Status	Notes
Customer self-registration	✓	<code>POST /api/users/register</code> works
Security hole: register accepts arbitrary <code>role</code>	⚠ FIX	Anyone can register as <code>manager</code> right now. Must force <code>role: customer</code> server-side.
Login / JWT	✓	Works, role embedded in token
<code>GET /me</code>	✓	Works
Manager-only staff creation	✗	No <code>POST /api/staff</code> exists. Currently staff would have to self-register with a role, which is the same security hole.
Staff listing/removal (<code>GET/DELETE /api/staff</code>)	✗	Missing entirely
<code>host</code> role	✗	Not in the <code>User</code> role enum at all — you have customer/waiter/chef/cashier/manager, no host. Spec requires host as a separate role from waiter.
Dead <code>isKitchenStaff</code> middleware	⚠ FIX	Defined, unused — remove or wire in
Role scoping to restaurant (a waiter at Restaurant A shouldn't touch Restaurant B's orders)	✗	Depends on multi-tenancy above

3. Menu

Item	Status	Notes
<code>Menu</code> / <code>MenuItem</code> model	✗	Doesn't exist. <code>Order.items</code> is just an embedded <code>{name, price, quantity}</code> — no real catalog.
Menu CRUD (manager)	✗	Missing
Availability toggle per item	✗	Missing
Customer-facing <code>GET /api/menu</code>	✗	Missing — customers have zero read access to anything menu-related right now
Categories	✗	Missing

4. Tables & Floor Plan

Item	Status	Notes
Basic <code>Table</code> CRUD (manager)	✓	Exists
<code>number</code> , <code>capacity</code> , <code>status</code> fields	✓	Exists
Spatial position (<code>gridX</code> , <code>gridY</code>)	✗	No spatial data on <code>Table</code> at all
<code>shape</code> field	✗	Missing
<code>combinable</code> flag	✗	Missing
<code>adjacentTo</code> (adjacency graph for combining)	✗	Missing — current override logic doesn't check adjacency at all, just grabs any available tables by capacity
<code>FloorElement</code> model (walls/windows/doors)	✗	Doesn't exist
<code>POST /api/floor-layout</code> (bulk save)	✗	Missing
<code>GET /api/floor-layout</code> (read for builder/host/customer views)	✗	Missing
Block deleting/resizing a table with active future reservations	✗	No such check exists — and can't exist meaningfully until <code>Reservation</code> model exists (see below)

5. Reservations / Booking (time-aware)

Item	Status	Notes
<code>Reservation</code> model with <code>timeSlot</code>	✗	Big gap. Current <code>Table.status</code> (available/reserved/occupied/cleaning) has no time dimension — can't represent "reserved for 8pm, free until then."
Customer self-booking route	✗	Only staff-driven <code>allocateTable</code> exists (walk-in style). No customer-initiated future booking.
Distinguish walk-in seating vs future reservation	✗	Currently conflated — <code>allocateTableService</code> only handles "seat now"
Reservation duration / overlap check	✗	Missing — needed to check two bookings don't overlap on same table
Reservation status lifecycle (locked→confirmed→seated→completed→cancelled→no_show)	✗	Only <code>Table.status</code> exists, no reservation-level state machine
No-show handling (host marks after grace period)	✗	Missing

6. Concurrency / Locking

Item	Status	Notes
Lock before confirm (TTL-based)	✗	Current allocation is direct find-and-save, no lock step — fine for single-host sequential use, not safe for concurrent customer self-booking
DB-level unique constraint backstop (<code>(table, timeSlot)</code>)	✗	Missing — needs the <code>Reservation</code> model to exist first
Re-validate status at confirm time (not just at lock time)	✗	Missing
Real-time broadcast of status changes (Socket.io)	✗	Entire stack is REST-only right now, no websocket layer

7. Table Combination Logic

Item	Status	Notes
Basic override — grab biggest available tables until capacity met	⚠️	Exists (<code>managerOverrideAllocate</code>) but...
Adjacency-aware (only combine physically adjacent tables)	❌	Current logic ignores adjacency entirely
Anti-waste ranking (smallest overshoot, fewest tables)	❌	Current logic is pure greedy-by-capacity-desc, no ranking/optimization
Auto-suggest + host-approval flow (vs fully manual manager trigger)	❌	Currently 100% manual — manager has to call override themselves, there's no auto-suggested combination surfaced to a host for approval

8. Waitlist

Item	Status	Notes
<code>WaitingQueue</code> model	✅	Exists
FIFO join on no-fit	✅	Exists
Auto-match on table free	✅	Exists in <code>freeTableService</code>
Skip-incompatible (not strict head-of-line blocking)	⚠️ VERIFY	Looks like it scans and first-fits, but confirm it doesn't block behind an incompatible front-of-queue entry — needs a read-through of the actual loop
Notify customer + response window before moving to next	❌	No notification mechanism exists at all (no SMS/push/email anywhere in the stack)
Host dashboard view of queue with position/wait time	❌	<code>GET /api/allocation/waiting</code> returns raw data, no live host UI (frontend gap, but also no "position/wait time" computed field)

9. Orders / Kitchen / Billing

Item	Status	Notes
Order creation (staff-side)	✓	Exists
Kitchen queue view/update	✓	Exists
Billing view/pay	✓	Exists
Bug: table freed twice (both <code>updateStatus</code> completion and <code>markPaid</code>)	⚠️ FIX	Redundant, needs single source of truth
Customer-side order placement / pre-order while booking	✗	No customer order route exists — only manager/waiter can create orders
Order tied to <code>Reservation</code> (not just <code>Table</code>)	✗	Currently orders link only to <code>Table</code> , not to a specific reservation/visit — fine for walk-in-only model, will need rethinking once reservations exist

10. Real-time layer

Item	Status	Notes
Socket.io or equivalent	✗	Nothing — entire backend is request/response only
Broadcast table status changes	✗	Missing
Broadcast kitchen order updates	✗	Missing (nice-to-have, not core)

11. Frontend (all of it)

Item	Status	Notes
Manager dashboard	✗	Doesn't exist — backend only, per your architecture doc
Floor plan grid-builder UI	✗	Doesn't exist
Customer booking flow UI	✗	Doesn't exist
Host live floor view	✗	Doesn't exist
Waiter/chef views	✗	Doesn't exist
Staff management UI	✗	Doesn't exist

Summary — what's genuinely solid already

- Auth/JWT plumbing, password hashing
- Role-check middleware pattern (needs a couple additions/fixes, not a rebuild)
- Basic table CRUD
- Order → kitchen → billing lifecycle for **walk-in, staff-driven** flow
- Waiting queue FIFO logic (core idea is right, needs verification + notification layer)

Summary — what's the real remaining work, roughly in dependency order

1. Multi-tenancy (Restaurant model + scoping) — foundational, blocks everything else
2. Auth fixes (lock down register, add staff creation route, add host role)
3. Menu model + routes
4. Reservation model (time-aware booking) — foundational for concurrency, combination, waitlist notify
5. Locking/concurrency layer
6. Floor plan spatial data + FloorElement model + layout endpoints
7. Combination logic rewrite (adjacency + anti-waste + approval flow)
8. Real-time (Socket.io)
9. Waitlist polish (verify skip-logic, add notification)
10. Bug fixes (double table-free, dead middleware)
11. Customer-side order placement
12. Entire frontend

This is the full list — nothing scored, no priority yet. Next step is deciding what's MVP-critical vs what can slip to phase 2, and sequencing the actual build.